권용재 Web Developer

서비스를 만들고 운영하면서, 더 나은 방법을 찾아가는 개발자입니다.



유한책임회사 티지나래 · 2024.06 ~ 재직 중

02-08쪽
실무 경험

09-12쪽
개인 프로젝트

13-15쪽
교육·팀 프로젝트

16쪽
기술·교육·자격

실무 경험 · SQL 성능 개선

조회 결과를 유지하며 A/S 조회 성능 개선

교육용 단말 약 11만 대의 배송·설치·A/S·점검 이력을 관리하는 시스템입니다. 기간을 지정해도 A/S 결과 조회가 느려 SQL 구조와 검색 조건을 바꿨습니다.

담당
조회 SQL 재작성 · 운영 DB 결과 비교·검증

A/S 완료월 조회 · 운영 DB 비교

53초 → 1.2초

01 통합 뷰 대신 필요한 이력 직접 조회
기본 테이블을 조인하고, 접수별 최신 이력을 직접 찾도록 SQL을 재작성했습니다.

02 날짜 컬럼의 함수 적용 제거
검색 조건을 시작일 이상·종료일 다음 날 미만의 범위 비교로 바꿨습니다.

03 검색 조건별로 기존 결과와 대조
운영 DB에서 조회값·NULL·중복 행과 컬럼 정보를 비교해 기존 결과가 유지되는지 확인했습니다.

전체 조회도 함께 개선
약 125초 → 44초

약 48만 건의 전체 조회도 운영 DB에서 비교했습니다. 월별 조회뿐 아니라 전체 조회에서도 기존 결과가 유지되는지 확인했습니다.

백엔드·DB
Java 11 · Spring MVC · MyBatis · MariaDB

프론트엔드
JavaScript · WebSquare5 · JSP

포트폴리오 사이트

실무 경험 · 콜센터 조회 성능 개선

콜센터 상담 중 반복되던 조회 대기시간 단축

고객을 선택하거나 서비스 접수를 마친 뒤 학교 서비스 이력이 늦게 표시됐습니다. 전체 접수·이력을 먼저 집계하던 SQL에서 병목을 찾았습니다.

담당

목록·건수 SQL 개선 · 결과 비교·검증 · 운영 반영

조회 API 응답 중앙값

20초 → 0.02초

운영 반영 전후 일별 로그 기준 · 근삿값

01 검색에 맞는 접수부터 찾기

학교·고객명·전화번호 조건으로 대상 접수를 좁힌 뒤 해당 접수의 최신 이력만 조회했습니다.

02 건수 조회의 불필요한 집계 제거

최신 이력을 모두 집계하지 않고 존재 여부만 확인했습니다. 묶음 접수의 건수와 검색·권한·페이징 조건은 유지했습니다.

03 기존 결과와 대조한 뒤 운영 반영

조회값·NULL·정렬·건수를 비교했습니다. 원본 이름 검색이 제한 시간을 넘긴 경우에는 범위나 조인 순서를 제한해 결과를 따로 확인했습니다.

상담사 피드백
접수 후 번호 표시 10초 → 1초

운영 반영 후 접수번호가 화면에 표시되기까지의 대기가 줄었다는 피드백을 받았습니다. 상담사가 체감한 근삿값입니다.

백엔드·DB

Java 11 · Spring MVC · MyBatis · MariaDB

프론트엔드

JavaScript · WebSquare5 · JSP

실무 경험

미완료 점검을 다음 방문까지 이어서

종이 점검표를 전산화하고, 단말별 미완료 사유와 이전 기록을 다음 방문에서도 확인하도록 개발했습니다.

담당

화면·API·SQL 개발 · 운영 데이터 정리·검증

실제 업무에 사용된 점검 기능

71개교 · 약 1만 5천 건의 점검 기록 저장

01 **점검 회차·결과 기록**

02 **미완료 사유를 다음 회차로 전달**

03 **서명·확인서와 결과 파일 발급**

종이 점검표와 재방문 업무 전산화

단말마다 점검 회차·결과·미완료 사유를 기록해 다음 방문에서 이어서 확인하도록 개발했습니다. 전자서명과 확인서 발급을 연결하고 여러 담당자가 수정할 때 다른 사람의 결과를 덮어쓰지 않도록 처리했습니다.

기존 점검 결과를 보존하며 중복 데이터 정리

14·15자리 일련번호가 섞여 다른 단말에 연결되거나 중복 저장되는 문제를 해결했습니다. 저장 기준을 통일하고 백업·복구·검증 절차를 준비한 뒤 기존 점검 결과를 남기면서 중복 행을 정리했습니다.

상담이력의 중복 요청과 이전 검색 결과 표시 방지

조회 요청을 한 번에 하나로 제한하고 검색 조건이 바뀌면 이전 응답이 화면에 표시되지 않도록 했습니다. 연속 스크롤·검색 조건 변경·오류 후 재조회 상황을 화면 함수 테스트로 검증했습니다.

엑셀 단말 등록 검증과 QR 발급

등록 목록을 생산입고 정보와 대조해 중복 일련번호와 기존 학교·사용자 정보를 확인하도록 개발했습니다. 검증을 통과한 단말을 일괄 등록하고 QR을 발급해 운영팀의 등록·출력 업무를 지원했습니다.

외부 연동 변경에 대응하는 공통 처리 정리

여러 화면에 흩어진 외부 API 호출과 인증 설정을 공통 모듈로 옮겼습니다. 연동 정보가 바뀌면 공통 코드와 설정에서 반영하도록 했습니다.

백엔드·DB

Java 11 · Spring MVC · MyBatis · MariaDB

프론트엔드

JavaScript · WebSquare5 · JSP

포트폴리오 사이트

실무 경험 · 비동기 파일 처리

압축 요청과 파일 다운로드를 분리해 진행 상태 제공

기존 문제 첨부파일 300~400건을 압축하는 동안 요청이 오래 유지되고, 진행 상황을 알기 어려웠습니다.

01 · 요청 접수

작업 번호 먼저 반환

브라우저에 작업 번호를 저장해 새로고침 후에도 같은 작업을 찾습니다.

02 · 별도 작업 실행

비동기 파일 압축

Spring 비동기 실행기에서 압축하고 진행·완료·실패 상태를 남깁니다.

03 · 화면에서 확인

작업 상태 조회

화면이 작업 번호로 상태를 주기적으로 조회해 진행 상황을 표시합니다.

04 · 완료 후 요청

압축 파일 다운로드

완료된 파일을 별도 요청으로 내려받고 임시 파일은 보관 기준에 따라 정리합니다.

Hazelcast로 작업 상태 공유

같은 호스트의 두 WAS 인스턴스가 작업 번호별 상태를 공유합니다. 상태 조회가 다른 인스턴스로 들어와도 진행 상황을 확인할 수 있도록 했습니다.

요청이 끝난 뒤에도 이어지는 작업

압축 처리와 화면 요청을 분리하고, 새로고침 후에도 기존 작업 번호로 진행 상황을 이어서 확인하도록 했습니다.

백엔드·DB
Java 21 · Spring Boot · MyBatis · MariaDB · Oracle

프론트엔드
JavaScript · Vue 2

상태 공유·배포
Hazelcast · Jenkins

포트폴리오 사이트

실무 경험 · 운영 오류와 배포

화면 상태와 요청 경계를 확인하며 오류 수정

본사와 약 500개 협력사가 등록·계약과 현장 엔지니어의 교육·자격·증빙을 관리하는 포털입니다. 초기 구축부터 화면·API·SQL 개발과 운영·배포에 참여했습니다.

담당

Vue 상태 분리 · 서버 검증 · 인증 요청 제어 · Jenkins 배포

01 **오류가 남는 화면 상태 추적**

02 **서버에서도 잘못된 저장 차단**

03 **동시 요청·실패 상황 검증**

이전 엔지니어의 첨부파일이 연결되는 오류

얕은 복사로 Vue 인스턴스들이 같은 첨부파일 정보를 참조한 것이 원인이었습니다. 화면마다 상태 객체를 새로 만들고, 서버에서도 다른 재직 엔지니어의 파일 연결을 막았습니다.

인증번호 반복 발송과 재사용 방지

Hazelcast에 사용자별 발송 제한을 공유해 동시에 들어온 요청 중 하나만 처리했습니다. DB에서 인증번호 확인과 사용 처리를 함께 수행하고, 인증번호와 문자 내용은 API 응답에서 제외했습니다. 동시 요청·만료·저장 실패 상황은 단위 테스트로 검증했습니다.

반복 배포 절차를 Jenkins 작업으로 연결

소스 반영·빌드·서버 전송·백업·두 WAS 인스턴스 순차 배포를 하나의 작업으로 연결했습니다. 개발·운영 작업을 구분하고 단계별 실행 기록을 남겼습니다.

백엔드·DB
Java 21 · Spring Boot · MyBatis · MariaDB · Oracle

프론트엔드
JavaScript · Vue 2

상태 공유·배포
Hazelcast · Jenkins

실무 경험

계정·알림 업무를 관리 화면으로

계정 변경과 사내 시스템 연동, 알림톡·계약서 발송, 교육 명단 취합 기능을 개발했습니다. 처리 전 검증과 실패 이력을 함께 제공해 운영자가 결과를 확인하도록 했습니다.

담당

초기 구축 참여 · Vue 화면·Spring Boot API·MyBatis SQL 개발

01 **업무 조건·중복 여부 검증**

02 **계정·부서 정보와 수신자 연계**

03 **처리 결과·실패 이력 확인**

본사 계정 발급·수정 업무 통합

계정 생성·수정과 사내 시스템 전송을 관리 화면에서 처리하도록 개발했습니다. 부서·등급·계정 중복을 검사하고 전송 실패도 화면에서 확인하도록 했습니다.

계정 변경 전 남은 업무 확인과 부서 정보 동기화

부서 변경·사용 중지 전에 남은 재고·서비스 업무를 검사하고 잔여 업무가 있거나 조회에 실패하면 저장을 막았습니다. 부서 변경 시 연동 시스템의 사용자·요청 정보에도 같은 부서코드를 반영했습니다.

알림톡 발송 정책·수신자 관리 기능 개발

발송 여부·메시지 양식·추가 수신자와 변경 이력을 관리하도록 개발했습니다. 기본·추가 수신자에 같은 엔지니어가 포함되면 한 번만 발송하고 발송 정책에 따른 처리를 단위 테스트로 확인했습니다.

완료 계약서 이메일 발송과 이력 관리

계약이 완료됐는지 확인한 뒤 문서를 첨부해 센터장에게 발송하도록 개발했습니다. 성공·실패와 오류 내용을 남겨 운영자가 발송 결과를 확인하도록 했습니다.

센터별 교육 명단 취합·검증

명단 제출부터 관리자의 참석 현황 확인·엑셀 다운로드까지 개발했습니다. 다른 센터 소속이거나 이미 등록된 엔지니어는 등록에서 제외할 대상으로 안내하고 제출을 마친 센터의 명단으로 현황을 집계했습니다.

백엔드·DB
Java 21 · Spring Boot · MyBatis · MariaDB · Oracle

프론트엔드
JavaScript · Vue 2

상태 공유·배포
Hazelcast · Jenkins

포트폴리오 사이트

실무 경험

조회 조건을 화면부터 서비스·SQL까지 연결

서비스 접수·처리, 자재 재고와 업무 현황을 관리하는 사내 시스템입니다. 입사 초기에는 Nexacro 화면과 Java 서비스·SQL의 기능 추가 및 오류 수정을 맡았습니다.

담당

Nexacro 업무 화면 개발 · Java 서비스·MyBatis SQL 추가·수정

01 **기간·제품군별 서비스 실적 조회**

02 **검색 오류와 재고 날짜 조건 수정**

03 **접속 이력·설치·철거 화면 개발**

기간·제품군별 서비스 실적 조회 개발

조회 기간에 맞춰 월별 열을 구성하고 제품군별 서비스 건수와 인도·반송·취소 현황을 집계했습니다. Nexacro 화면, Java 서비스와 MyBatis SQL을 연결해 기간별 실적을 비교하도록 했습니다.

전화번호로 서비스 내역을 찾지 못하는 오류 수정

화면 입력값과 서버 전달값의 연결을 수정하고 암호화된 전화번호를 조회하는 SQL 조건을 추가했습니다. 일반전화·휴대전화와 보조 연락처로 관련 서비스 내역을 찾도록 처리했습니다.

재고 조회의 날짜 조건과 검색 분기 수정

기준일 재고와 현재 재고를 비교하는 화면의 조회 조건을 보완했습니다. 자재 입출고 내역의 날짜 컬럼에 적용하던 형변환을 제거하고 현재 재고 선택 여부에 따라 집계 범위를 구분했습니다.

접속·개인정보 다운로드 이력 조회 개발

기존 로그를 부서·사용자·기간별로 집계하는 SQL과 조회 화면을 추가했습니다. 로그인, 화면별 접속, 개인정보 다운로드 횟수를 메뉴에서 확인하도록 했습니다.

우정사업 설치·철거 등록 화면과 SQL 추가

장비 관리번호로 대상 정보를 조회하고 설치·철거 내용을 저장하는 기능을 개발했습니다. 작업 구분에 맞춰 입력 항목과 대상 조회를 연결했습니다.

백엔드·DB

Java 8 · Spring MVC(eGovFrame 3.9) · MyBatis · Oracle

프론트엔드

JavaScript · Nexacro 17

포트폴리오 사이트

개인 프로젝트

오늘사이

오프라인 로테이션 소개팅 서비스

오프라인 로테이션 소개팅의 참가자와 운영자를 위한 서비스를 개발하고 있습니다. 회원·인증, 일정 관리, 참가 신청·취소까지 구현했으며 현장 진행·매칭은 이후 단계입니다.



현재 구현 범위: 회원·인증 · 일정 관리 · 참가 신청·취소

모집 정원과 동시 신청 처리

회차별 DB 행 잠금을 사용하고 정원 확인과 신청 저장을 한 트랜잭션에서 처리했습니다. 회원 조회를 대체한 로컬 테스트에서 마지막 자리 1개에 20건이 동시에 신청해도 1건만 성공하는지 확인했습니다. 중복 신청과 모집 마감·정원 변경 중 신청도 테스트했습니다.

참가자·운영자 화면과 취소 규칙 구현

참가자는 신청 내역을 조회·취소하고 운영자는 회차별 신청 목록을 관리합니다. 회차 취소 시 해당 신청을 함께 취소하고 이미 접수된 인원보다 정원을 줄이지 못하게 했습니다.

화면과 서버를 연결한 신청 흐름 검증

참가자·운영자 화면과 서버를 함께 실행해 Playwright로 검증하고 GitHub Actions에서 자동 실행합니다. 회원 서비스 중단 시 오류 안내와 로그인 유지, 복구 후 재신청을 확인했습니다.

백엔드·DB
TypeScript · NestJS · Prisma · PostgreSQL

프론트엔드
TypeScript · React · Next.js

테스트·자동화
Vitest · Playwright · GitHub Actions

포트폴리오 사이트 GitHub 저장소

오늘사이 · 업무 규칙과 동시 신청

참가자와 운영자의 요청이 같은 정원 규칙을 따르도록

참가자 웹

일정 확인 신청 조건 안내 **신청·취소**

로그인 상태, 모집 마감, 정원과 기존 신청 여부에 따라 신청 가능 여부를 안내합니다.

운영자 웹

회차 관리 신청 목록 확인 **취소·정원 변경**

회차별 신청 상태를 조회하고, 신청 취소와 모집 정원 변경을 처리합니다.

서버의 신청 처리 하나의 트랜잭션

회차 행 잠금 → **정원·신청 조건 검사** → **신청 저장** → **커밋·잠금 해제**

같은 회차의 정원 확인과 신청 저장이 동시에 엇갈리지 않도록 처리했습니다.

마지막 자리의 중복 신청 방지

마지막 자리 1개에 동시 신청 20건을 보내 1건만 성공하는지 확인했습니다.

회차와 신청의 취소 상태 일치

회차를 취소하면 해당 회차의 신청도 함께 취소하도록 했습니다.

접수된 인원보다 정원 축소 금지

정원을 바꾸는 동안 들어온 신청과 모집 마감 중 신청도 테스트했습니다.

동시 신청 수치는 회원 조회를 대체한 로컬 테스트 결과입니다. 현장 진행·매칭은 이후 구현할 계획입니다.

백엔드·DB
TypeScript · NestJS · Prisma · PostgreSQL

프론트엔드
TypeScript · React · Next.js

테스트·자동화
Vitest · Playwright · GitHub Actions

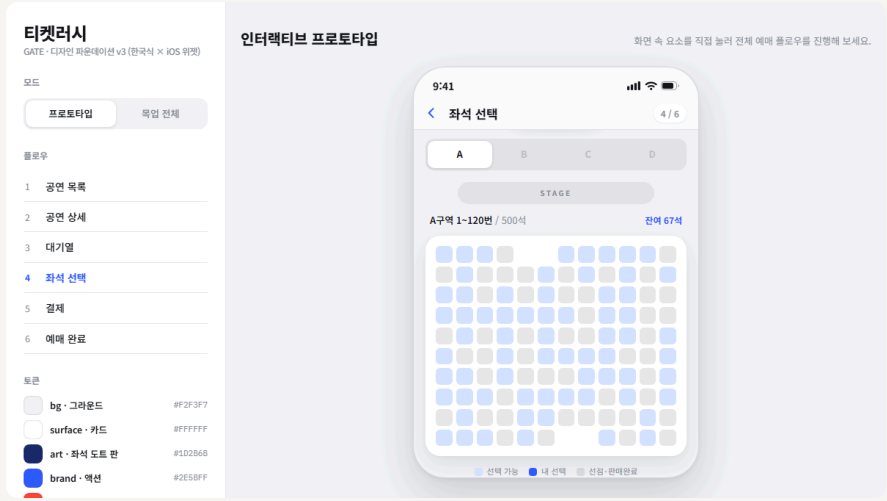
포트폴리오 사이트 GitHub 저장소

개인 프로젝트

티켓러시

선착순 티켓 예매 서비스

공연 조회, 대기열 입장, 좌석 선택, 모의 결제와 예매 완료까지 구현한 개인 프로젝트입니다. 같은 좌석을 여러 사용자가 동시에 선택하거나 결제를 반복 요청하는 상황을 중점적으로 다뤘습니다.



결제는 모의 처리하며, 동시 요청 수치는 로컬 테스트 결과입니다.

좌석 선점과 최종 예약의 중복 방지

Redis로 좌석을 5분간 선점하고 MySQL의 회차·좌석 복합 기본키로 최종 예약의 중복을 막았습니다. MySQL·Redis를 사용하는 로컬 통합 테스트에서 같은 좌석에 선점 요청 100건을 동시에 보내 1건만 성공하는지 검증했습니다.

결제 접수와 결과 반영 분리

예약을 결제 진행 상태로 바꾸는 작업과 결제 요청 이벤트 저장을 하나의 트랜잭션으로 처리했습니다. 같은 예약에 확정 요청 10건이 동시에 들어와도 이벤트 1건만 남도록 상태 검사와 낙관적 잠금을 적용했습니다.

대기열·좌석·결제 상태를 웹 화면에 연결

Next.js로 예매 흐름을 만들고 SSE로 대기 순번과 좌석 상태를 갱신했습니다. 결제 접수 후에는 결과를 조회해 승인·거절을 표시하고 응답 유실 시 같은 요청 식별자로 재시도하도록 처리했습니다.

백엔드·DB
Java 21 · Spring Boot · MySQL · Redis

프론트엔드
TypeScript · React · Next.js

테스트·자동화
Playwright · GitHub Actions

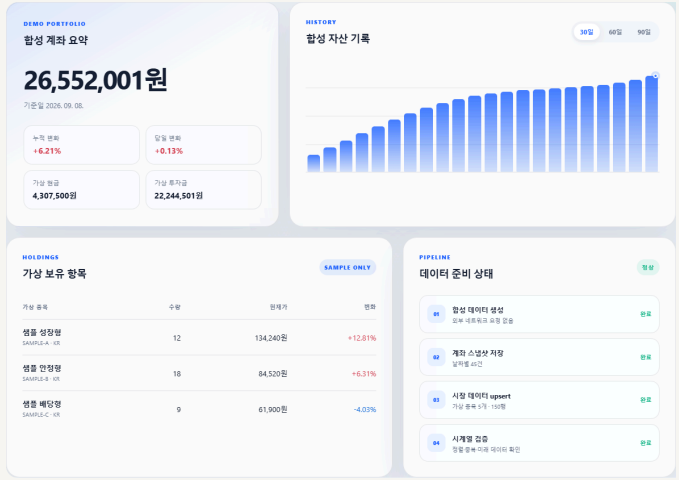
포트폴리오 사이트 GitHub 저장소 직접 예매해 보기

개인 프로젝트

ReachRich

개인 투자 연구·운영 플랫폼

토스증권 API로 계좌·시세를 수집하고 투자 전략을 검증·모의 운용하는 개인 프로젝트입니다. 데이터 수집부터 결과를 확인하는 대시보드까지 연결했습니다.



화면은 합성 데이터 예시이며, 설명은 비공개 원본 구현을 기준으로 합니다.

계좌·시세 데이터의 중복 없는 적재

SQLAlchemy·SQLite로 날짜별 계좌 요약과 보유 종목을 한 트랜잭션에서 교체 저장했습니다. 일봉은 Parquet에 날짜별로 병합해 재수집 시 중복을 막았습니다.

외부 API 호출 제한과 부분 실패 처리

OAuth2 토큰은 만료 전까지 재사용하고 호출 제한 응답에는 지정된 시간 뒤 한 번 재시도합니다. 일부 종목이 실패해도 수집을 이어가되 실패 비율이 기준을 넘으면 작업을 실패로 처리합니다.

미래 데이터 혼입을 막는 전략 검증

기존 검증 모듈을 옮겨 전략에는 판단 시점까지의 데이터만 전달하고 매매 결정은 다음 시점부터 반영했습니다. 이후 데이터를 제거해도 과거 신호가 바뀌지 않는지 검사합니다.

계좌 현황과 매매일지를 연결한 대시보드

FastAPI로 저장 데이터를 조회하고 React에 자산 추이·보유 종목·매매일지를 연결했습니다. 금액 가리기 설정을 저장하고 숨겨진 탭에서는 주기적인 조회를 멈췄습니다.

실행 환경을 나눈 자동 작업과 실패 알림

등록 IP가 필요한 수집은 로컬에서, 모의 운용·상태 점검은 GitHub Actions에서 실행합니다. 실패는 텔레그램으로 알리고 로컬 작업은 다음 단계를 이어갑니다.

백엔드·데이터
Python · FastAPI · SQLAlchemy · SQLite · Parquet

프론트엔드
TypeScript · React · Vite

테스트·자동화
pytest · Vitest · GitHub Actions

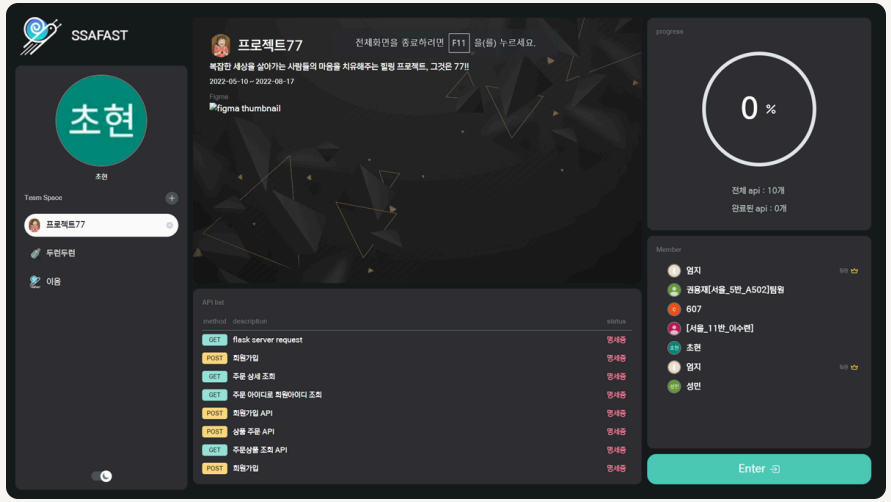
포트폴리오 사이트

팀 프로젝트 · 6인

SSAFAST

API 명세·테스트 협업 도구

API 명세를 작성하고 요청을 실행해 응답과 성능 테스트 결과를 확인하는 팀 협업 도구입니다. 프론트엔드에서 명세 입력 폼과 테스트 요청·결과 화면을 맡았습니다.



반복·중첩 구조의 API 입력 폼 구현

React Hook Form으로 헤더·경로 변수·쿼리와 중첩 객체를 입력받는 폼을 구성했습니다. API 명세에 따라 입력 항목을 표시하고 테스트에 사용할 값을 관리했습니다.

입력값을 테스트 요청으로 변환

폼의 헤더·경로 변수·쿼리·본문을 요청 데이터로 조합하고 실행 API와 연결했습니다. 성능 테스트의 초당 요청 횟수·지속 시간을 입력받아 필수값과 허용 범위를 검사했습니다.

테스트 이력과 결과 상세 화면 개발

이전 실행 결과를 조회하고 응답 지연 시간의 평균·백분위, 처리량, 상태 코드별 응답 수를 표시했습니다. 명세 입력에서 실행·결과 확인까지 한 작업 흐름으로 연결했습니다.

프론트엔드

TypeScript · React · Next.js · React Hook Form

포트폴리오 사이트 GitHub 저장소 API 테스트해 보기

팀 프로젝트

ddoing

그림으로 배우는 영어 단어

영어 단어를 보고 그림을 그리면, 팀이 만든 AI 판정 결과에 따라 학습이 이어지는 팀 프로젝트입니다.



이미지 전처리는 팀장과 함께 진행했고, AI 모델 학습은 팀장이 담당했습니다.

Canvas 그림 입력과 AI 판정 연동

화면에 그린 그림을 이미지로 바꿔 AI 판정 서버에 보내고 판정 결과에 따라 점수와 경험치를 반영했습니다.

중복 실행되던 타이머 수정

화면에 들어올 때 타이머가 두 번 돌던 문제를 고치고 화면을 벗어나면 타이머도 멈추게 했습니다.

다음 문제로 넘어갈 때 상태 초기화

다음 문제로 넘어갈 때 이전 단어 목록이 남지 않도록 상태 갱신 시점을 고쳤습니다.

담당 범위

메인·그림 학습 화면 · 그림 입력과 AI 판정 서버 연동 · 학습 상태·점수·타이머 처리

프론트엔드

TypeScript · React · Vite

입력·상태

Canvas API · Redux Toolkit

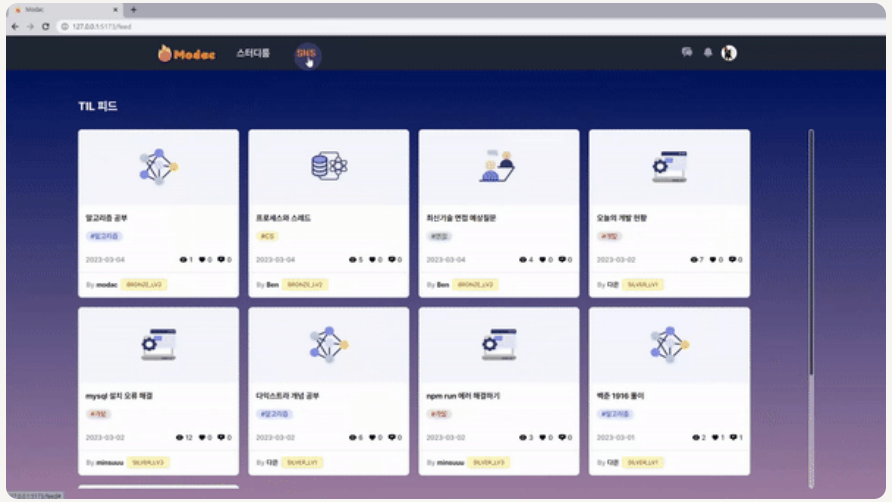
포트폴리오 사이트 GitHub 저장소 직접 그려 보기

팀 프로젝트 · 6인

MODAC

개발자 학습 기록·스터디 서비스

개발자가 학습 기록을 남기고 스터디를 함께 운영하는 서비스를 만든 팀 프로젝트입니다.



팀이 구성한 SockJS·STOMP 연결을 스터디룸 화면에 연동했습니다.

스터디룸 입장·퇴장과 초대 코드 검증

Vue 3와 Pinia로 스터디룸 입장·퇴장, 초대 코드 검증, 화면 상태를 관리했습니다.

학습 기록과 개인화 화면 개발

학습 기록과 통계, 알림, 즐겨찾기 화면을 만들었습니다.

SockJS·STOMP 채팅 화면 연결

연결 상태와 새로 받은 메시지를 스터디룸 채팅 화면에 보여 줍니다.

담당 범위

스터디룸·게시글·마이페이지 화면 · 학습 통계·알림·즐겨찾기 · 채팅 화면 연결

프론트엔드

Vue 3 · Pinia

채팅 연동

SockJS · STOMP

포트폴리오 사이트 GitHub 저장소 스터디 참여해 보기

실무에서 사용한 기술

백엔드

Java · Spring Boot · Spring MVC · MyBatis

프론트엔드

JavaScript · Vue 2 · WebSquare5 · Nexacro 17 · JSP

데이터베이스

SQL · MariaDB · Oracle

배포·형상관리

Jenkins · Git · SVN

개인·팀 프로젝트의 기술은 각 프로젝트에 표시했습니다.

교육·자격·어학

2026.06 — **TOEIC 885점**
LC 480 · RC 405

2026.06 — **TOEIC Speaking Advanced Low**
170점

2024.09 — **SQLD**
SQL 개발자 자격 취득

2022.07 ~ 2023.06 — **삼성 청년 SW 아카데미(SSAFY) 8기**
웹 개발 과정 수료. 팀 프로젝트 세 개에서 프론트엔드를 맡았습니다.

2018.03 ~ 2020.08 — **아주대학교 e-비즈니스학과**
학사 졸업

2014.01 ~ 2015.05 — **California State University, Chico**
Business Administration 전공 후 아주대학교 편입